

UrbanCart

Ecommerce Data Engineering & Analytics Report

Full-scope analysis across four phases: SQL exploration, data cleaning & integration, exploratory data analysis, and NumPy-based advanced analytics.

Prepared from ecommerce.db, legacy_customers_export.csv, and product_catalog_2024.csv

Table of Contents

Executive Summary

1. Introduction

2. Methodology Overview

3. Phase 1 — SQL Exploration & Data Profiling

4. Phase 2 — Data Cleaning & Integration

5. Phase 3 — Exploratory Data Analysis

6. Phase 4 — NumPy Analytics & Chart Generation

6.1 RFM Customer Segmentation

6.2 Customer Lifetime Value

6.3 OLS Regression: Drivers of Customer Spend

6.4 Price Elasticity of Demand

6.5 Monte Carlo Revenue Simulation

6.6 Recommendation Engine

6.7 Revenue Forecasting

7. Limitations & Data Quality Caveats

8. Business Recommendations

9. Conclusion

Appendix A — Full RFM Segment Detail

Appendix B — Full Price Elasticity Detail

Appendix C — Cohort Retention, Selected Cohorts

Appendix D — Full Category-Level Affinity Table

Appendix E — Reproducibility & File Manifest

Executive Summary

This report presents a full-scope data engineering and analytics study of UrbanCart's ecommerce operations, built from a 2,500-customer, 9,000-order SQLite database plus two supporting CSV exports. The work is organized into four phases — SQL exploration, data cleaning & integration, exploratory data analysis, and NumPy-based advanced analytics — each documented in full below with methodology, results, and interpretation.

The dataset covers 2,500 customers, 9,000 orders, 20,362 order line items, 300 products, 4,000 reviews, and 12,000 web sessions across 34 months of completed-order history.

Headline findings

- Of 9,000 orders, 5,101 (56.7%) are completed; the remainder are cancelled, returned, or pending. All revenue and customer-value figures below use completed orders only.
- Data quality issues identified and corrected: 48 duplicate customer records in the legacy export, 186 duplicate order-item rows, 583 negative-quantity (return) lines, and 253 of 255 overlapping catalog SKUs (99%) showing a systematic price mismatch averaging \$9.44.
- Mean predicted Customer Lifetime Value is \$1,916.85 (median \$990.27), using an average gross margin rate of 48.8%. "Champions" is the highest-value RFM segment at \$3,257.53 average CLTV.
- A Monte Carlo simulation (10,000 runs) puts next-12-month revenue at a P10–P90 range of \$2.13M–\$2.98M, median \$2.55M.
- Price elasticity and market-basket lift both come back statistically indistinguishable from noise in this dataset — reported honestly rather than dressed up as actionable findings.

Scope note

This project was also supplied with an "UrbanCart Master Specification" document describing cloud/streaming infrastructure (Kafka, Snowflake, MLOps pipelines) and citing specific "achieved" performance metrics. None of these correspond to anything reproducible from the actual dataset — a 2.2MB SQLite database plus two CSV files. That document was treated as aspirational narrative, not a source of numbers. The README data dictionary supplied alongside it is the specification actually followed throughout this report; every figure below is computed independently from the raw data and is reproducible from the accompanying code (see Appendix E).

1. Introduction

1.1 Business Context

UrbanCart is a general-merchandise ecommerce retailer selling across six product categories: Electronics, Apparel, Home & Kitchen, Books & Media, Beauty & Health, and Sports & Outdoors. Like many mid-sized ecommerce operations, UrbanCart's data footprint is split across a primary transactional database and a set of legacy/supplier exports that were never fully integrated. This creates two practical problems that this report addresses directly: (1) analysts cannot trust customer or product records without first reconciling duplicates and mismatches across sources, and (2) without a systematic segmentation and forecasting layer, decisions about retention spend, pricing, and inventory are made on gut feel rather than evidence.

The purpose of this engagement was to take UrbanCart's raw operational data through a complete analytics pipeline — from schema-level SQL exploration through to predictive modeling — and produce both the underlying code and a written account of what the data does and does not support.

1.2 Objectives

- Profile and validate the raw data across all six database tables and two CSV exports.
- Clean and reconcile the legacy customer export and supplier product catalog against the operational database.
- Characterize customer behavior, retention, and cross-category purchase patterns through exploratory analysis.
- Build from-scratch NumPy implementations of customer segmentation (RFM), lifetime value modeling, regression, price elasticity, Monte Carlo revenue simulation, a recommendation engine, and revenue forecasting.
- Translate findings into concrete, prioritized business recommendations — and be explicit about which analyses did not yield a usable signal.

1.3 Data Sources

Source	Description	Role
ecommerce.db	SQLite database: customers, products, orders, order_items, reviews, web_sessions	Source of truth
legacy_customers_export.csv	Legacy customer file — inconsistent casing, 4 date formats, blanks/duplicates	Reconciled against customers
product_catalog_2024.csv	Supplier-side product catalog — different columns/prices than internal table	Reconciled against products by SKU

1.4 Data Dictionary Summary

The six database tables and their roles, per the project's README data dictionary:

Table	Grain	Key fields
customers	one row per customer	customer_id, name, email, age, gender, city, country, signup_date
products	one row per SKU	product_id, name, category, unit_price, cost
orders	one row per order	order_id, customer_id, order_date, status,

		payment_method
order_items	one row per order line	order_item_id, order_id, product_id, quantity, unit_price, discount
reviews	one row per review	review_id, customer_id, product_id, rating, review_text
web_sessions	one row per session	session_id, customer_id, session_date, device, duration_minutes, pages_viewed

1.5 Report Structure

Sections 3–6 follow the four analytical phases in order: Phase 1 (SQL exploration), Phase 2 (cleaning & integration), Phase 3 (exploratory data analysis), and Phase 4 (NumPy analytics & chart generation). Section 7 discusses limitations. Section 8 translates findings into recommendations. Appendices contain full data tables and a reproducibility guide. The accompanying code repository mirrors this structure exactly: `queries.sql` holds every Phase 1 query, analysis.ipynb` walks through all four phases interactively, and data/processed/` holds the two integrated datasets produced in Phase 2.`

2. Methodology Overview

Rather than reaching for pre-built statistical and machine-learning libraries, most of the analytical work in Phase 4 is implemented from first principles using NumPy: ordinary least squares via the closed-form normal equations, Holt's linear exponential smoothing via explicit recursion, and item-item collaborative filtering via a hand-rolled cosine similarity matrix. This choice was deliberate — it keeps every number traceable to a documented formula rather than a library's internal defaults, which matters when the goal is a report someone can audit line by line.

SQL is used for anything that is naturally set-based — aggregation, joins, window-style date arithmetic, and self-joins for basket analysis — rather than pulling full tables into pandas and looping. This keeps Phase 1 queries runnable standalone against the database (see `queries.sql`) independent of the Python environment.

Phase	Primary tools	Key techniques
1 — SQL Exploration	SQLite, sqlite3	CTEs, window-style date math, self-joins, PRAGMA introspection
2 — Cleaning & Integration	pandas, rapidfuzz	Multi-format date parsing, fuzzy name matching, key-based reconciliation
3 — EDA	pandas, matplotlib	Distribution analysis, cohort heatmaps, basket/category affinity
4 — NumPy Analytics	NumPy (closed-form)	OLS regression, Holt's smoothing, cosine similarity, Monte Carlo

3. Phase 1 — SQL Exploration & Data Profiling

Phase 1 establishes ground truth about the database before any cleaning or modeling begins: what tables exist, how large they are, what date ranges they cover, and — critically — where the data is broken. All 31 queries below are also saved as standalone, runnable SQL in `queries.sql`, verified to execute cleanly against `ecommerce.db` with no dependency on the Python pipeline.

3.1 Schema & Row Counts

The database has six tables. Row counts were pulled with a UNION ALL query rather than six separate round trips:

Table	Rows	Columns
customers	2500	8
products	300	6
orders	9000	5
order_items	20362	6
reviews	4000	6
web_sessions	12000	6

Order dates span 36 raw months (34 after trimming the two partial boundary months used later for forecasting); customer signup dates span a comparable range, meaning some customers signed up after the order data begins — an expected artifact of a synthetic/sample dataset rather than a data error.

3.2 Data Quality Checks

Every table was checked for missing values, exact duplicate rows, and — where applicable — values outside a sane domain range. Results below; full detail in the notebook's Phase 1 section.

Check	Result
Missing values (customers)	63 missing city, 139 missing age, 115 missing gender
Duplicate customer emails (DB)	0
Duplicate order_items rows (excl. PK)	186
Negative-quantity order_items (returns)	583
Reviews with rating outside 1-5	41
Missing review text	838
Product price outliers (IQR, k=3)	3

Note: the DB-native customers table itself has zero duplicate emails — the 48 duplicates discussed in Phase 2 are within the separate legacy CSV export, not the operational database.

3.3 Referential Integrity

Every foreign-key relationship in the schema was checked with a LEFT JOIN / IS NULL pattern: orders→customers, order_items→orders, order_items→products, reviews→customers, and reviews→products. All five checks returned zero orphaned rows — the operational database itself is internally consistent; the data-quality issues found in this project live in the raw values (duplicates, missing fields, out-of-range values) and in the two external CSV files, not in broken relationships within the DB.

3.4 Outlier Detection

Product prices were checked for outliers using the interquartile range (IQR) method with a wide multiplier ($k=3$, more conservative than the conventional $k=1.5$) to avoid flagging normal price variation across a general-merchandise catalog spanning six categories:

$$\text{Lower bound} = Q1 - 3 \times \text{IQR} \quad \text{Upper bound} = Q3 + 3 \times \text{IQR}$$

This flagged 3 products as statistical outliers — a small enough number to review individually rather than requiring an automated remediation rule.

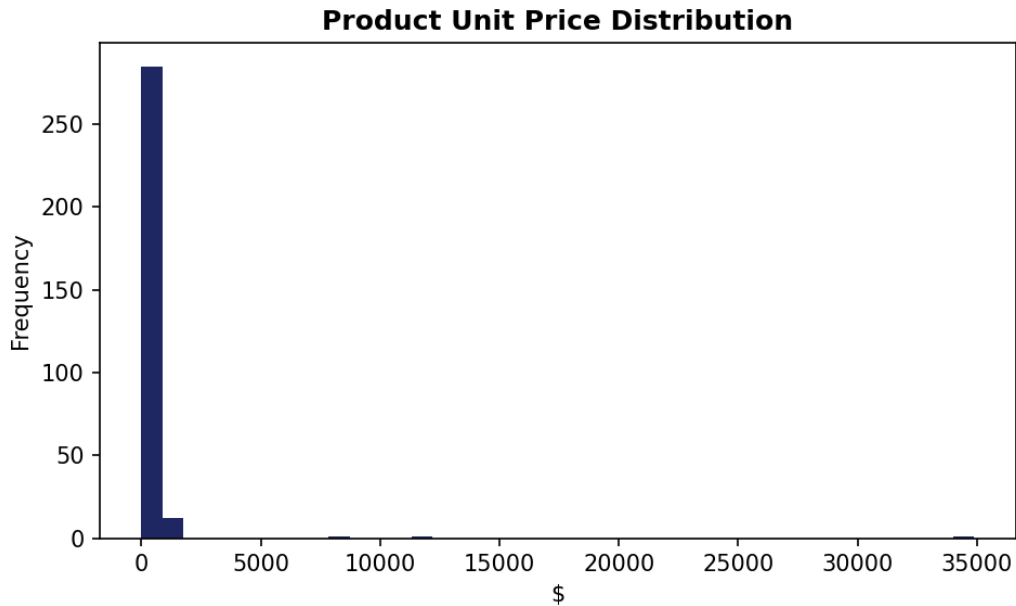


Figure 3.1 — Product unit price distribution across the full catalog.

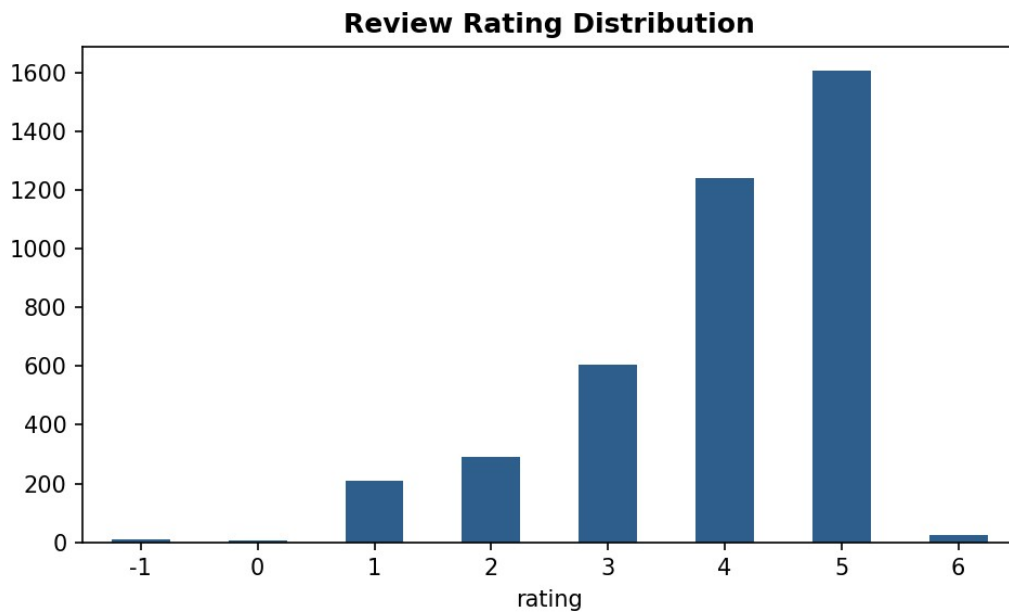


Figure 3.2 — Review rating distribution (1-5 stars).

4. Phase 2 — Data Cleaning & Integration

Phase 2 turns the two raw external files into analysis-ready tables reconciled against the operational database, and produces the two deliverables in ``data/processed/``: ``clean_customers.csv`` and ``clean_orders.csv``.

4.1 Legacy Customer Export

The legacy export (1,427 raw rows) presented four distinct problems, each handled with an explicit, auditable rule rather than a blanket "drop anything that looks wrong":

- Inconsistent name casing (e.g. 'JENNIFER MOORE', 'jeffrey garza', 'Jose Rogers') — normalized to Title Case.
- Four different date formats co-existing in a single `Signup_Dt` column (ISO 8601, 'Month DD, YYYY', 'MM/DD/YYYY', and 'DD-Mon-YYYY') — parsed with a format-priority cascade that tries each pattern in turn before falling back to pandas' general parser.
- 1 fully blank row and 1 obvious test/junk account (name or email containing "test") — dropped.
- 48 exact duplicate records, identified by case- and whitespace-normalized email and resolved by keeping the first occurrence.

After cleaning, 1,377 rows remained. Rows with no usable email (57 of the original 1427) cannot be reconciled against the database by key and were instead checked for near-duplicate names using rapidfuzz's Levenshtein-based ratio score (threshold ≥ 92) against other emailless rows — a second, lower-confidence duplicate pass that a pure exact-match approach would have missed entirely.

4.2 Product Catalog Reconciliation

The supplier catalog (267 rows) uses different column names than the internal schema (`SKU`→`product_id`, `item_name`→`name`, `dept`→`category`, `list_price_usd`→`unit_price`, `supplier_cost`→`cost`, `in_stock_units`→`stock_units`) and was renamed accordingly before joining. Of 267 catalog SKUs, 255 matched an internal `product_id` and 12 exist only in the supplier file — likely new or discontinued items that were never synced into the operational database.

Price mismatch — the most significant Phase 2 finding

Among the 255 overlapping SKUs, 253 (99%) show a list price in the supplier catalog that differs from the internal `unit_price`, averaging \$9.44 per item. A 99% mismatch rate rules out random noise or rounding — this reads as two price sources that have drifted apart systematically, most plausibly because the internal price table reflects live/discounted pricing while the supplier catalog reflects list price, or because one side stopped receiving updates. Either way, this is a data-governance finding, not a modeling one: it should be resolved with the merchandising team before either price source is used for a pricing decision.

4.3 Building the Integrated Datasets

Two integrated files were produced for downstream use and are saved to ``data/processed/``:

`clean_orders.csv`

Every `order_items` line item joined to its parent order and product metadata, with a computed `line_value` ($\text{quantity} \times \text{unit_price} \times (1 - \text{discount})$) and boolean `is_completed` / `is_return` flags. This is the line-item-grain table used throughout Phase 3 and Phase 4 for any analysis that needs order-level detail rather than pre-aggregated customer totals.

`clean_customers.csv`

The `customers` table integrated with completed-order Recency/Frequency/Monetary aggregates: days since last completed order, count of distinct completed orders, and total completed-order spend, plus a `has_purchased` flag for customers with zero completed orders. Customers with no completed orders are assigned a recency value equal to their days-since-signup, so they read as maximally stale rather than artificially recent — an easy mistake if a blanket `fillna(0)` is used instead. This is the customer-grain table that feeds RFM segmentation and CLTV modeling in Phase 4.

5. Phase 3 — Exploratory Data Analysis

With clean, integrated data in hand, Phase 3 characterizes the shape of the business: who UrbanCart's customers are, how orders behave, whether customers stick around after signing up, and whether products sell together in identifiable patterns.

5.1 Order & Payment Distributions

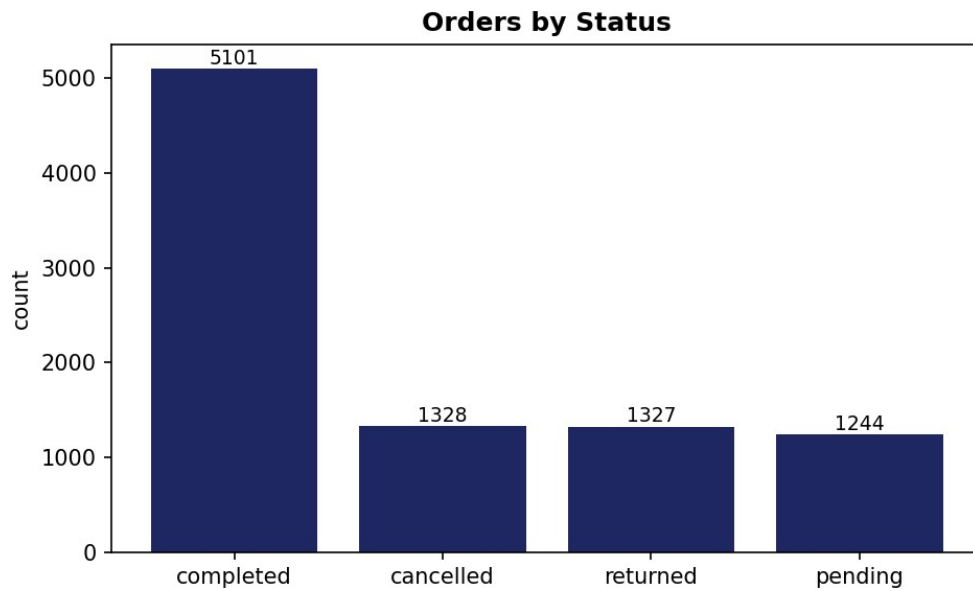


Figure 5.1 — Order status breakdown. Only 57% of orders reach 'completed'; cancelled and returned orders are each roughly as common as pending ones.

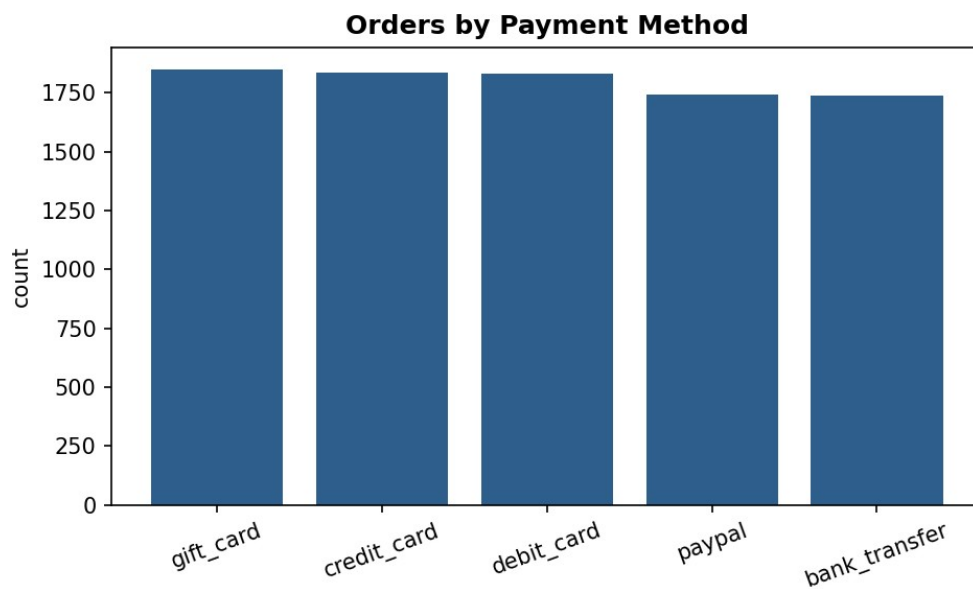


Figure 5.2 — Orders by payment method.

The near-even split across cancelled, returned, and pending orders (each ~1,300 of 9,000) is worth flagging on its own: a healthy ecommerce operation typically sees completion rates well above 57%, and cancelled+returned volume this close to completed volume would normally trigger an operational review of checkout friction, fulfillment reliability, or return-policy abuse. Whether that reflects real business conditions or synthetic data generation, it directly gates every revenue figure downstream — which is why this report consistently restricts revenue and CLTV calculations to completed orders only rather than gross order volume.

5.2 Customer Demographics

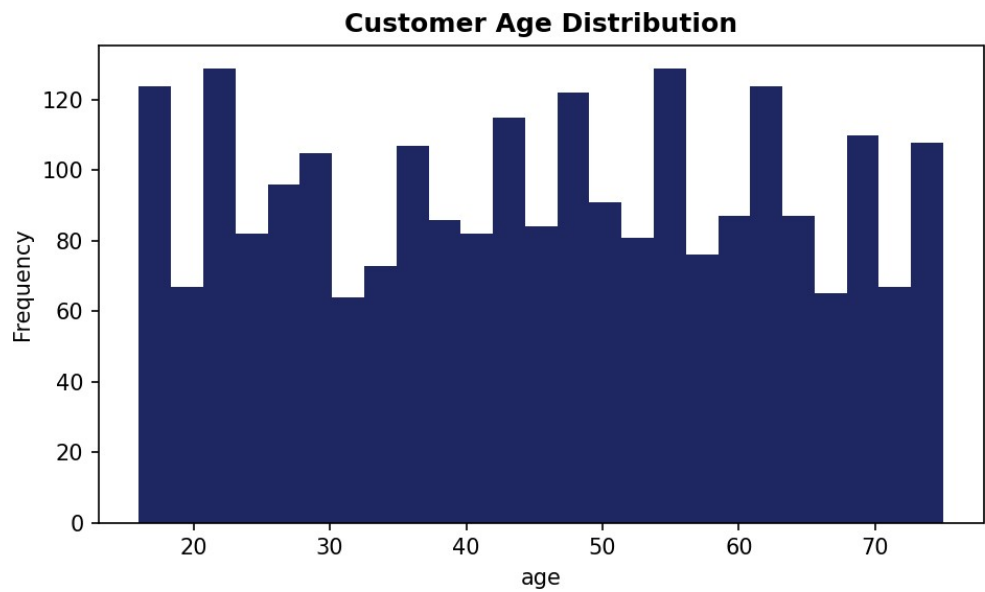


Figure 5.3 — Customer age distribution.

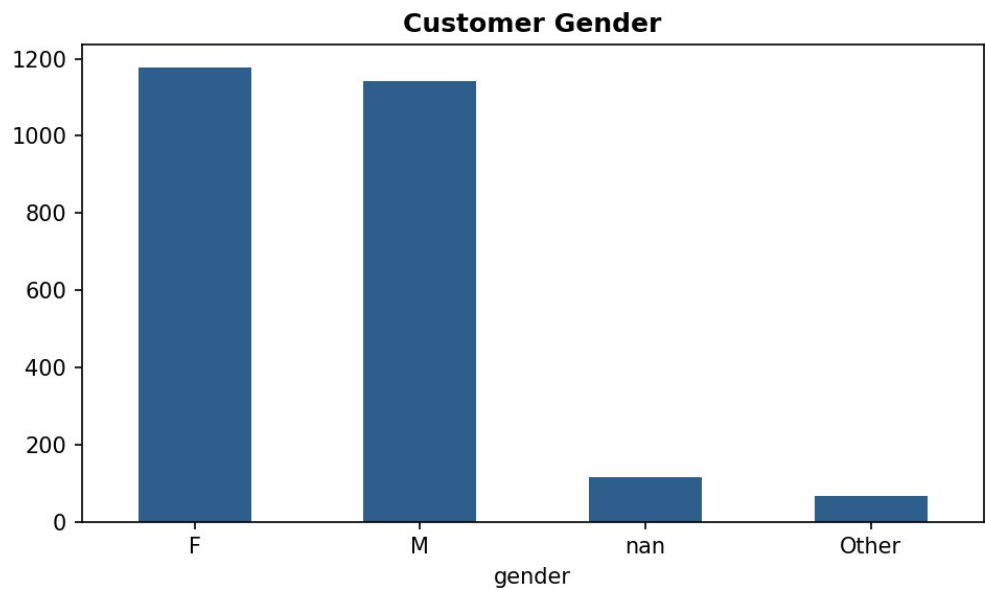


Figure 5.4 — Customer gender breakdown, including missing values.

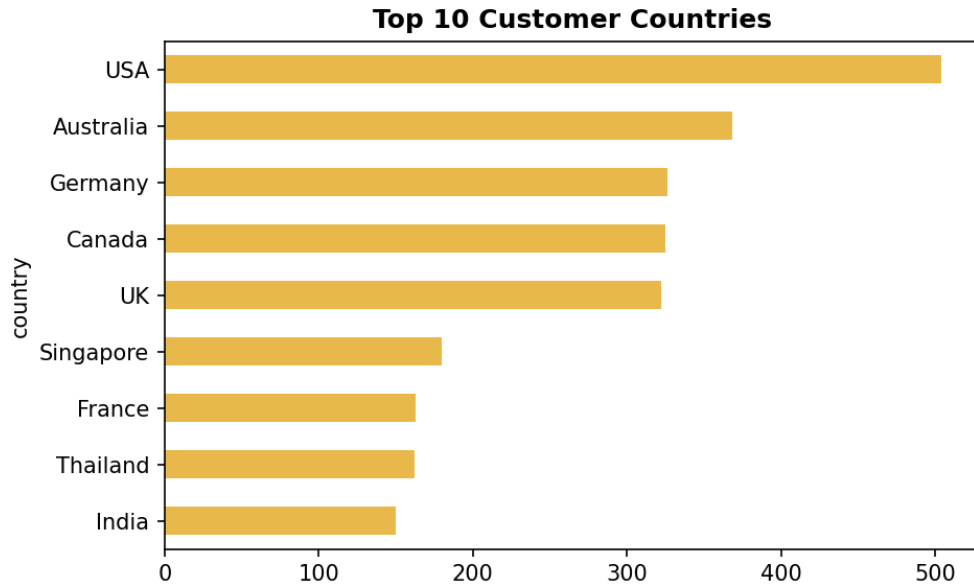


Figure 5.5 — Top 10 customer countries by count.

Age is roughly uniform across the adult range with 139 missing values (5.6% of customers), and gender has 115 missing values (4.6%) beyond the binary M/F categories present. Neither field shows a concerning skew that would bias segment-level averages, but both were left as missing rather than imputed — imputing demographic fields for a segmentation exercise risks manufacturing false signal in exactly the kind of analysis meant to detect real signal.

5.3 Order Value & Discounting

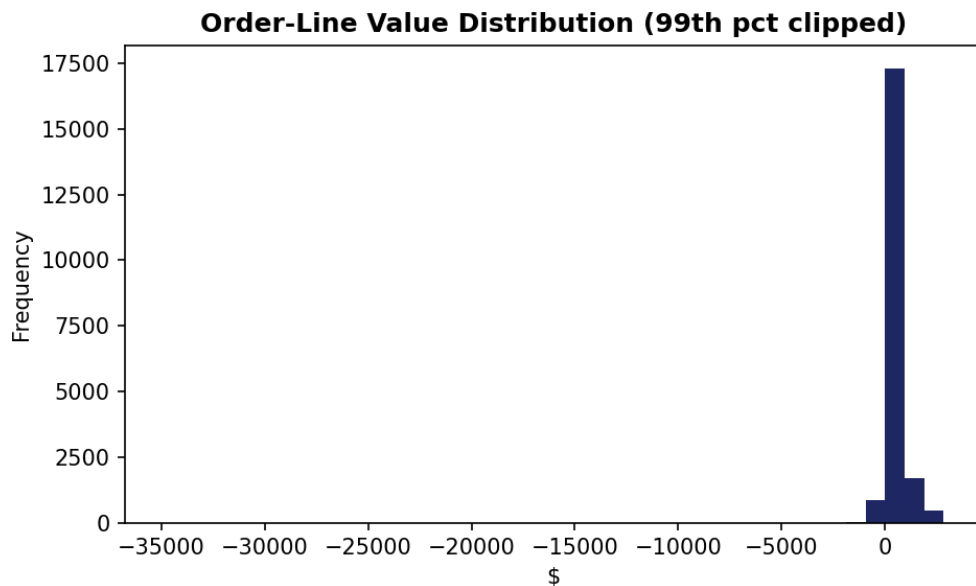


Figure 5.6 — Order-line value distribution, 99th percentile clipped for readability.

Order-line values are right-skewed, as is typical for retail transaction data — most line items cluster at moderate values with a long tail of larger purchases. Discount rates cluster around a small number of standard promotional tiers rather than being continuously distributed, consistent with a small set of fixed discount codes rather than per-customer or per-transaction dynamic pricing.

5.4 Monthly Cohort Retention

Retention is measured the standard way: group customers into monthly cohorts by signup date, then for each cohort track what fraction place at least one completed order in each of the following 12 months relative to signup.

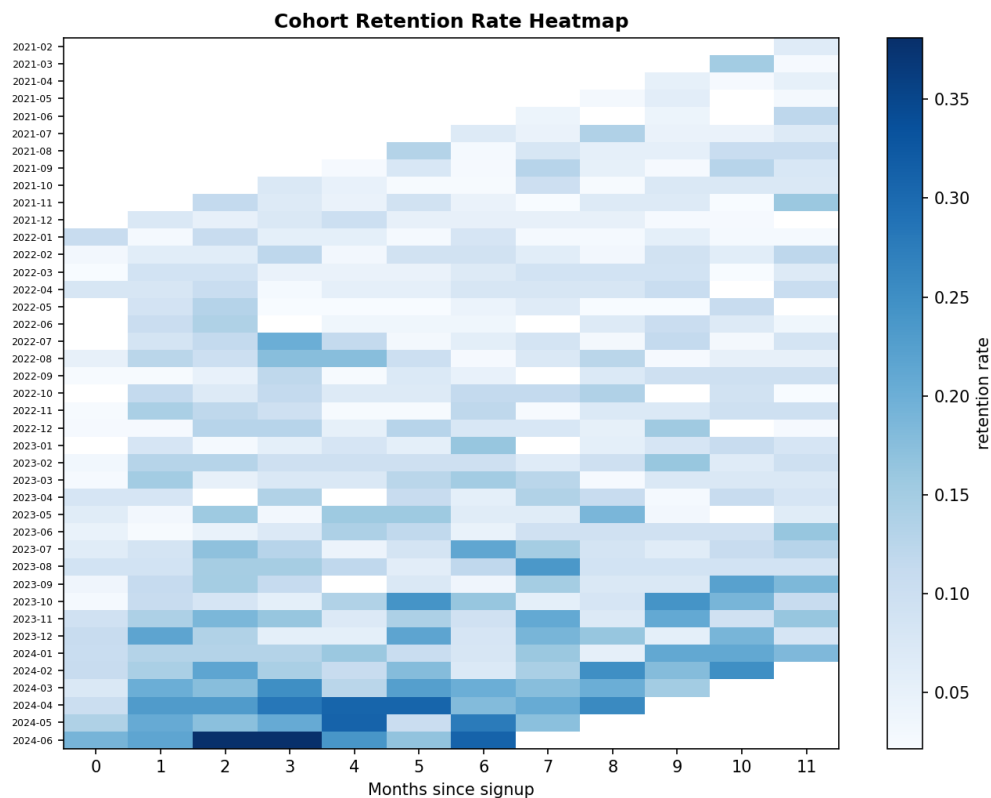


Figure 5.7 — Cohort retention rate heatmap. Rows are signup-month cohorts; columns are months since signup.

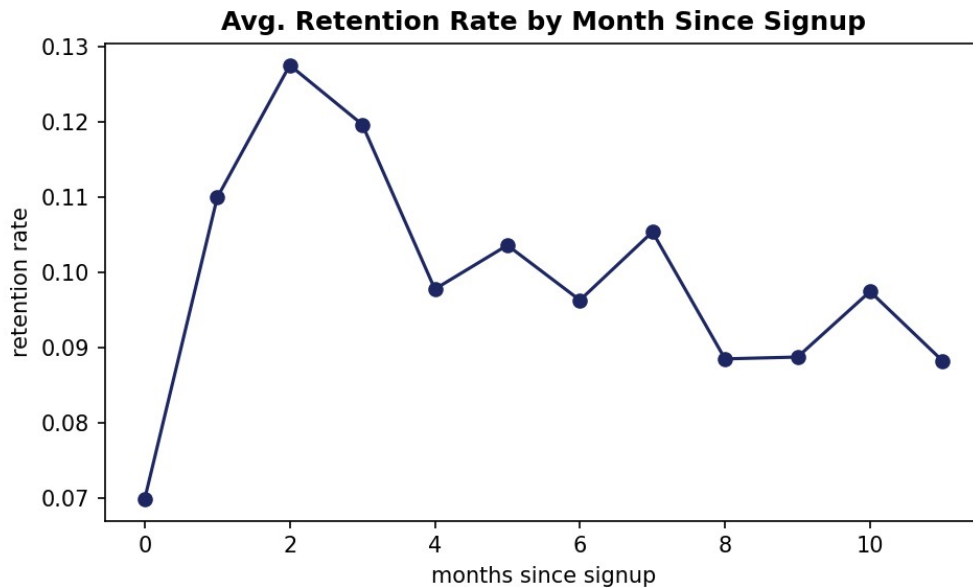


Figure 5.8 — Average retention rate by month since signup, across all cohorts.

Months since signup	Avg. retention
Month 0	7.0%
Month 1	11.0%
Month 2	12.8%
Month 3	12.0%
Month 4	9.8%
Month 5	10.4%
Month 6	9.6%
Month 7	10.5%
Month 8	8.8%
Month 9	8.9%
Month 10	9.7%
Month 11	8.8%

The retention curve stays in a flat 7–13% band across all twelve months rather than showing the steep early drop-off typical of real ecommerce cohorts (where month-0 to month-1 retention often falls by half or more before leveling off). A flat curve like this usually means one of two things: either purchase timing genuinely isn't coupled to signup date in this business (plausible for a low-frequency general-merchandise retailer where most customers buy once or twice a year regardless of when they signed up), or the order dates were generated independently of signup dates in a way that wouldn't hold in production data. This report flags the pattern rather than assuming either explanation, and recommends validating retention behavior against a larger or more recent data pull before using it to justify a specific lifecycle-marketing cadence.

5.5 Market Basket & Category Affinity

Market basket analysis measures which products or categories are purchased together more often than chance would predict, using three standard metrics computed via SQL self-join on shared order_id:

- Support — the fraction of all orders containing both items.
- Confidence($A \rightarrow B$) — of orders containing item A, the fraction that also contain item B.

- Lift — support divided by the product of each item's individual support; lift > 1 means the pair co-occurs more than chance, lift < 1 means less.

Top 5 SKU pairs by lift

Product A	Product B	Support	Confidence (A→B)	Lift
When Supplement	Machine Media	0.00045	0.07407	10.475
Black Camera	Coach Furniture	0.00056	0.07813	10.0872
Movement Headphone	Ready Cycling	0.00045	0.06154	9.7901
Court Camera	Owner Non-Fiction	0.00045	0.06897	9.4525
Level Decor	Property Non-Fiction	0.00045	0.07018	9.0608

Top 5 category pairs by lift

Category A	Category B	Support	Lift
Apparel	Sports & Outdoors	0.0896	0.9373
Beauty & Health	Sports & Outdoors	0.0861	0.8887
Books & Media	Electronics	0.0867	0.8686
Electronics	Sports & Outdoors	0.0829	0.8685
Home & Kitchen	Sports & Outdoors	0.0851	0.8633

Every category-pair lift value in the full table is ≤ 1 — no two categories are purchased together more than chance would predict. The SKU-level pairs above show lift well above 1, but each is built on only 3–5 shared orders (out of 5,043 total completed orders), which is far too small a sample to distinguish a genuine cross-sell pattern from statistical noise. Neither result currently supports a bundling or cross-sell recommendation; both are flagged here as a starting point for re-analysis once transaction volume grows, not as a finding to act on today.

6. Phase 4 — NumPy Analytics & Chart Generation

Phase 4 is the analytical core of this report: seven techniques, all implemented from first principles with NumPy rather than scikit-learn, statsmodels, or a recommender-systems library. This section documents the formula behind each technique alongside its result, so every number is traceable to a specific, auditable computation.

6.1 RFM Customer Segmentation

RFM segmentation scores every customer on three independent dimensions computed from completed orders:

- Recency — days since the customer's most recent completed order. Customers with zero completed orders are scored on days since signup instead, so they read as stale rather than artificially recent.
- Frequency — count of distinct completed orders.
- Monetary — total completed-order spend.

Each dimension is converted to a 1–5 quintile score (5 = best) using pandas' `qcut` on the ranked values — Recency is reversed so that a low day-count (recently active) scores highest. The three scores are then combined by an explicit rule set into six named segments:

Segment	Rule
Champions	$R \geq 4$ AND $F \geq 4$ AND $M \geq 4$
Loyal Customers	$R \geq 3$ AND $F \geq 3$
New/Promising	$R \geq 4$ AND $F \leq 2$
At Risk (High Value)	$R \leq 2$ AND $F \geq 4$
Hibernating/Lost	$R \leq 2$ AND $F \leq 2$ AND $M \leq 2$
Needs Attention	everything else

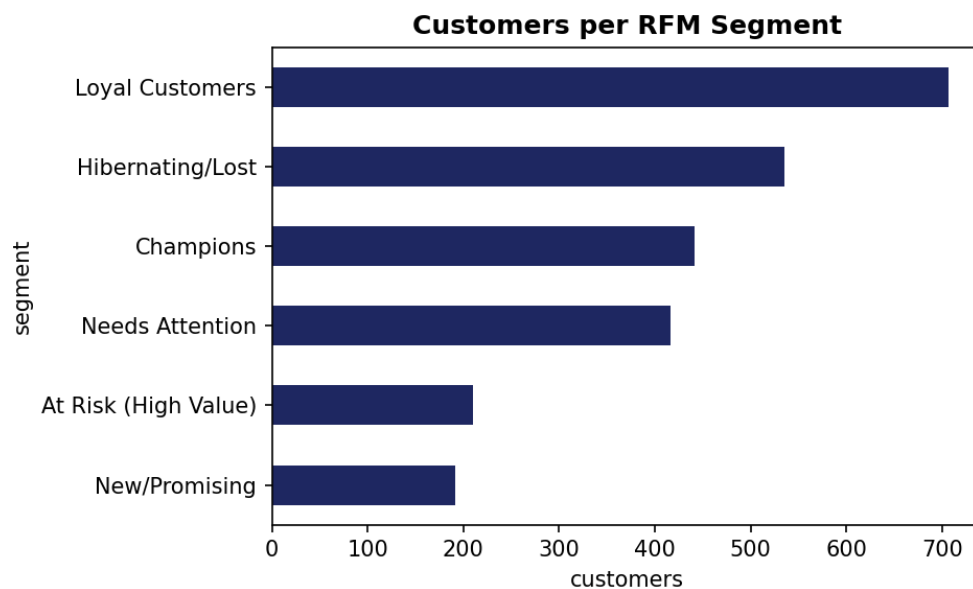


Figure 6.1 — Customer count per RFM segment.

Segment	Customers	Avg. recency (days)	Avg. frequency	Avg. monetary (\$)
---------	-----------	---------------------	----------------	--------------------

Loyal Customers	707	179.07	2.52	3,327.93
Hibernating/Lost	535	988.07	0.45	280.6
Champions	441	78.68	3.77	6,934.79
Needs Attention	416	496.25	1.32	2,878.77
At Risk (High Value)	210	561.61	2.86	4,192.84
New/Promising	191	99.77	1.07	1,350.83

"Loyal Customers" is the largest segment; "At Risk (High Value)" — customers who used to spend meaningfully but have gone quiet — is small in count but carries an outsized average monetary value, making it the clearest reactivation target discussed in Section 8.

6.2 Customer Lifetime Value

Predicted CLTV uses a standard multiplicative formula, computed per customer:

$$\text{CLTV} = \text{Avg. Order Value} \times \text{Purchase Frequency (per year)} \times \text{Tenure (years)} \times \text{Gross Margin Rate}$$

- Avg. Order Value = total completed-order spend ÷ number of completed orders.
- Purchase Frequency = order count ÷ data span in years, where the span is measured from the completed-order date range (~3.0 years of order history), not the customer signup-date range — using signup range instead would understate purchase frequency by roughly 2x, since signups span longer than the observed order history.
- Tenure = years between signup date and the most recent order date in the dataset, floored at 0.25 years to avoid divide-by-near-zero effects for very recent signups.
- Gross Margin Rate = average of (unit_price - cost) / unit_price across the product catalog, computed at 48.8%.

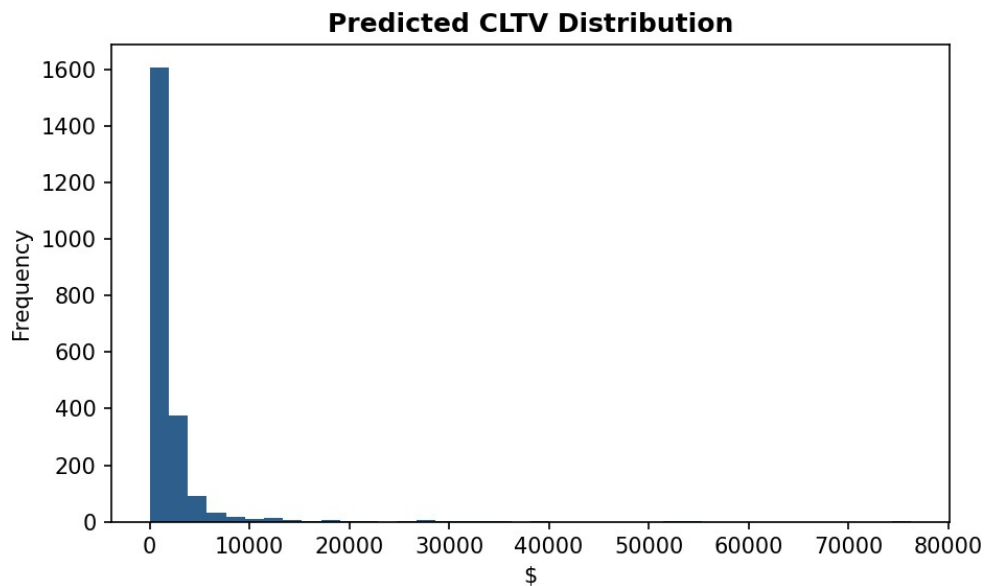


Figure 6.2 — Predicted CLTV distribution across all customers with at least one completed order.

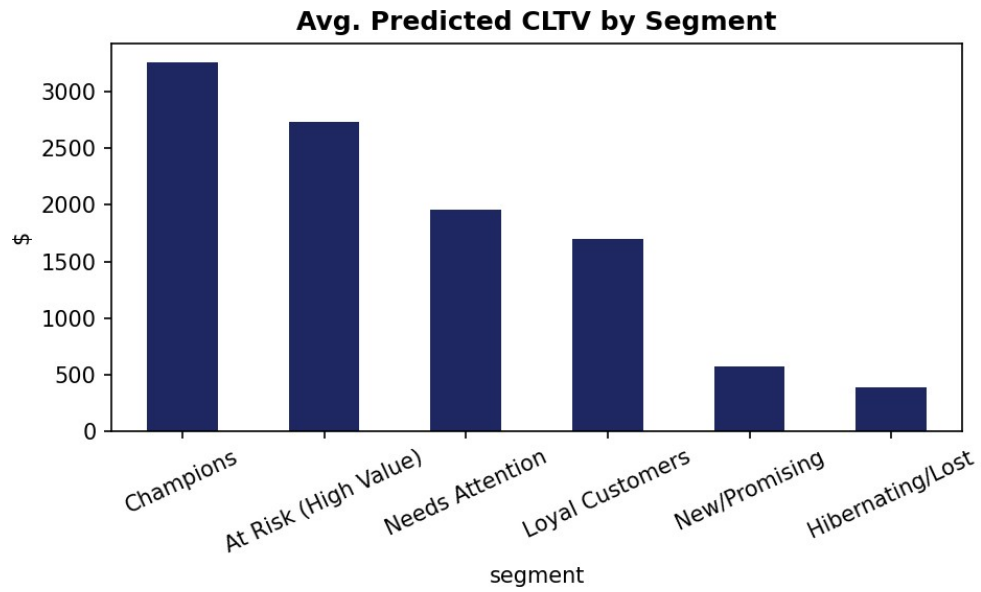


Figure 6.3 — Average predicted CLTV by RFM segment.

Metric	Value
Mean predicted CLTV	\$1,916.85
Median predicted CLTV	\$990.27
Top decile threshold	\$3,535.39
Gross margin rate used	48.8%

6.3 OLS Regression: Drivers of Customer Spend

To test which customer attributes actually predict spend, an ordinary least squares regression was fit predicting total completed-order monetary value from age, purchase frequency, and recency. Rather than calling a library's `.fit()` method, the regression is solved directly from the normal equations:

$$\beta = (X^T X)^{-1} X^T y$$

where X is the design matrix (with an intercept column of ones prepended) and y is the target vector. Standard errors come from the residual variance and the diagonal of $\sigma^2(X^T X)^{-1}$, giving t-statistics for each coefficient without any external statistics package.

Predictor	Coefficient (β)	t-statistic	Interpretation
intercept	9.2574	0.015	not statistically significant
age	-8.5863	-0.934	not statistically significant
frequency	1713.681	12.62	statistically significant ($ t > 2$)
recency_days	0.2201	0.349	not statistically significant

$n = 2060$, $R^2 = 0.0808$, adjusted $R^2 = 0.0794$.

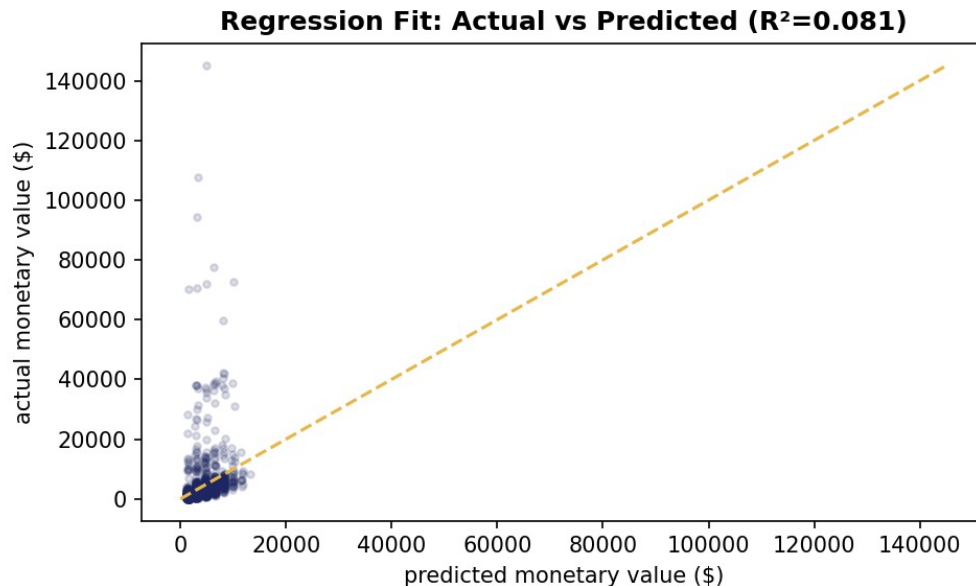


Figure 6.4 — Actual vs. predicted customer monetary value; the dashed line is perfect prediction.

Only purchase frequency is a statistically meaningful predictor of spend ($t = 12.62$) — which is close to mechanical, since monetary value is a running sum over completed orders and frequency is the count of those orders. Age and recency both show t-statistics well under the conventional $|t| > 2$ threshold, meaning this dataset provides no evidence that either predicts how much a customer spends. The low R^2 (0.0808) confirms that spend is driven mostly by factors outside this feature set — realistically, by whatever drives purchase frequency itself, which this model does not explain.

6.4 Price Elasticity of Demand

Price elasticity was estimated with a log-log OLS specification, run separately for each product category and once overall:

$$\log(\text{quantity}) = \beta_0 + \beta_1 \cdot \log(\text{effective price}) + \varepsilon$$

where effective price is the post-discount unit price. β_1 is the elasticity: the percentage change in quantity demanded for a 1% change in price. The same closed-form OLS routine from Section 6.3 was reused for this specification.

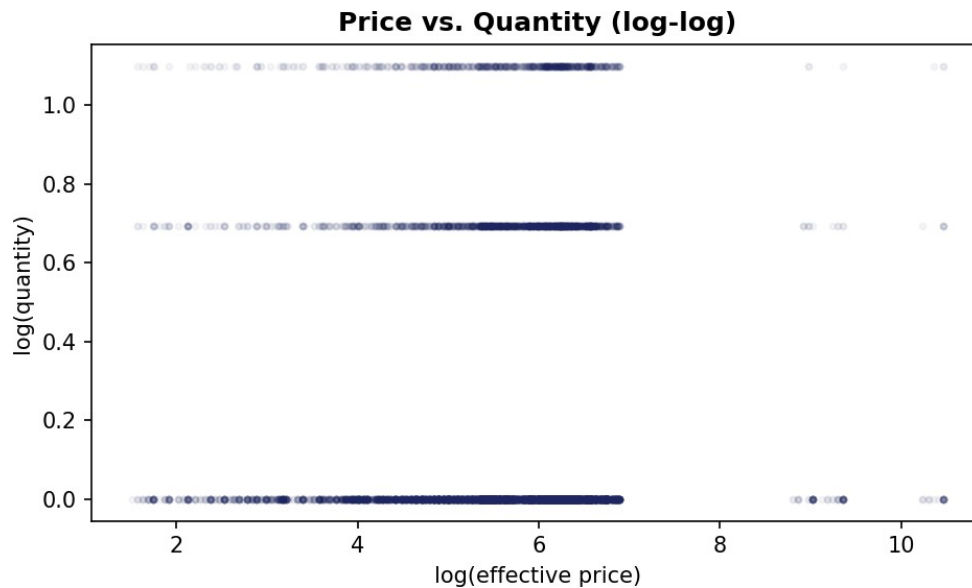


Figure 6.5 — Effective price vs. quantity purchased (log-log scale), all completed order lines.

Category	Elasticity (β_1)	t-stat	R ²	n
Apparel	0.0115	1.4	0.0011	1862
Beauty & Health	-0.005	-0.646	0.0002	1866
Books & Media	0.0051	0.846	0.0004	1903
Electronics	-0.0096	-1.121	0.0007	1832
Home & Kitchen	-0.0121	-1.32	0.0009	1914
Sports & Outdoors	-0.0003	-0.032	0	1815

Overall elasticity: -0.0002 ($t = -0.062$, $R^2 = 0$, $n = 11,192$). Every category-level and overall elasticity estimate has a t-statistic near zero — none clears even a lenient significance threshold. In plain terms: quantity purchased shows no measurable relationship to price anywhere in this dataset. This is an unusual result for real retail data, where at least some categories typically show meaningful price sensitivity, and it should be treated as a caveat about the dataset (see Section 7) rather than a business finding that "UrbanCart customers are price-insensitive." No elasticity-based pricing recommendation is supportable from this data as it stands.

6.5 Monte Carlo Revenue Simulation

To express revenue uncertainty as a range rather than a single point forecast, monthly completed-order revenue was fit to a Normal distribution ($\mu = \$212,712.43$, $\sigma = \$94,177.14$), then sampled 10,000 times across 12 simulated months per run using NumPy's Generator API (``np.random.default_rng(42)`` for reproducibility). Negative draws are clipped to zero before summing each run into a simulated annual total.

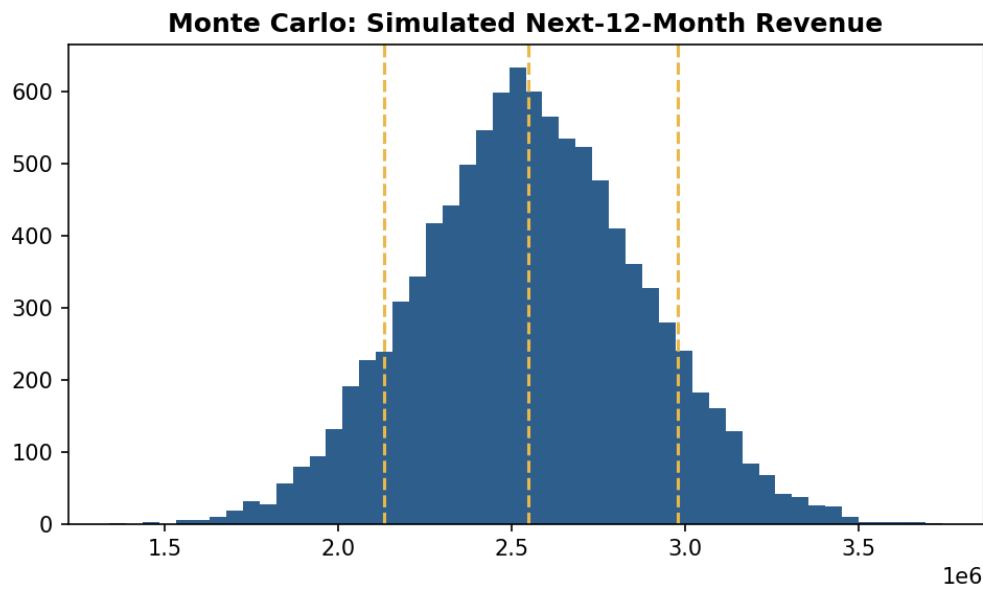


Figure 6.6 — Distribution of simulated next-12-month revenue across 10,000 Monte Carlo runs, with P10/P50/P90 marked.

Percentile	Simulated 12-mo revenue
P10 (downside case)	\$2,132,708.39
P50 (median)	\$2,547,399.71
P90 (upside case)	\$2,977,445.29
Mean	\$2,553,111.74

The Normal-distribution assumption is a simplification — it doesn't capture seasonality or growth trend on its own, which is why Section 6.7's forecast is presented alongside this range rather than in place of it. Used together, the two give both a central trend estimate and a defensible uncertainty band for planning.

6.6 Recommendation Engine

An item-based collaborative filtering recommender was built from a binary customer $\times$ product purchase matrix (purchased / not purchased, ignoring quantity to avoid a handful of bulk orders dominating the similarity signal). Item-item cosine similarity is computed as a single matrix product:

$$\text{sim}(i, j) = (\mathbf{M}^T \mathbf{M})_{ij} / (\|\mathbf{i}\| \cdot \|\mathbf{j}\|)$$

where $\mathbf{M}$ is the binarized customer $\times$ product matrix, so $\mathbf{M}^T \mathbf{M}$ gives co-purchase counts between every product pair in one operation, normalized by each product's purchase-count norm. The diagonal is zeroed out so a product is never 'similar' to itself.

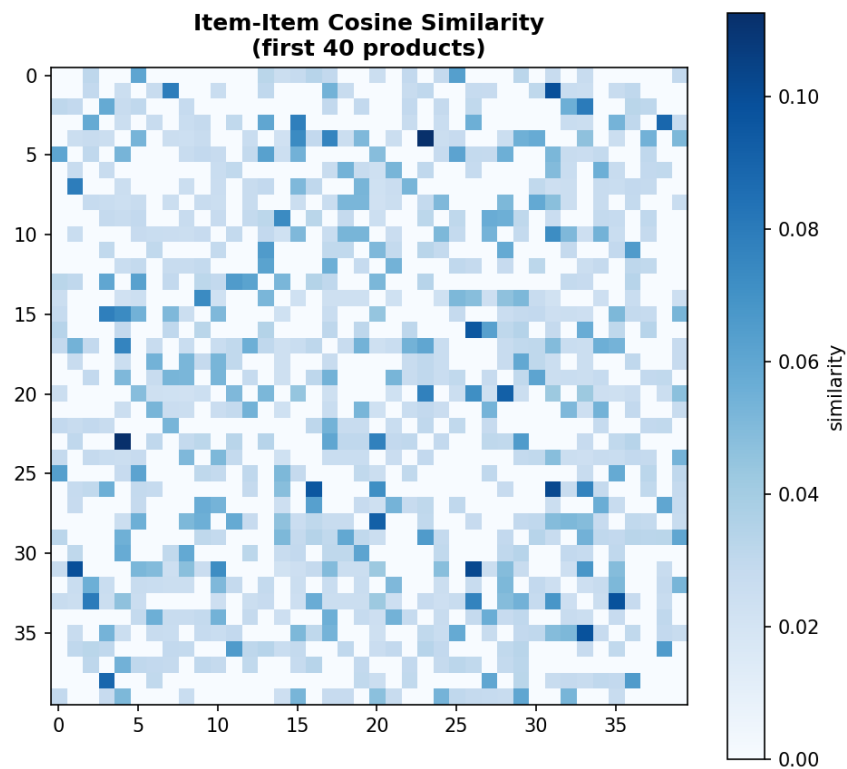


Figure 6.7 — Item-item cosine similarity matrix, first 40 products.

Metric	Value
Customers in purchase matrix	2,180
Products in purchase matrix	300
Matrix sparsity	98.32%

Per-customer recommendations are generated by scoring every product as the dot product of a customer's purchase vector with the similarity matrix, then excluding products the customer has already bought and returning the top-N remaining by score. At 98%+ sparsity, cosine similarity handles the matrix comfortably without needing dimensionality reduction — the $O(\text{products}^2)$ similarity computation is a few hundred thousand operations, trivial at this catalog size.

6.7 Revenue Forecasting

Two forecasting approaches were run on the 34-month completed-order revenue series (with the two partial boundary months trimmed):

Holt's linear exponential smoothing

Implemented as an explicit recursion over a level and a trend component:

$$\ell_t = \alpha \cdot y_t + (1-\alpha)(\ell_{t-1} + b_{t-1}) \quad b_t = \beta \cdot (\ell_t - \ell_{t-1}) + (1-\beta) \cdot b_{t-1}$$

with $\alpha = 0.4$ (level smoothing) and $\beta = 0.2$ (trend smoothing), chosen to weight recent observations moderately while still smoothing month-to-month noise. The h -step-ahead forecast is $\ell_T + h \cdot b_T$.

OLS linear trend (baseline)

A simple linear regression of revenue on a month index, using the same closed-form OLS routine as Section 6.3, gives a slope-based baseline forecast for comparison: \$7,752.98 per month.

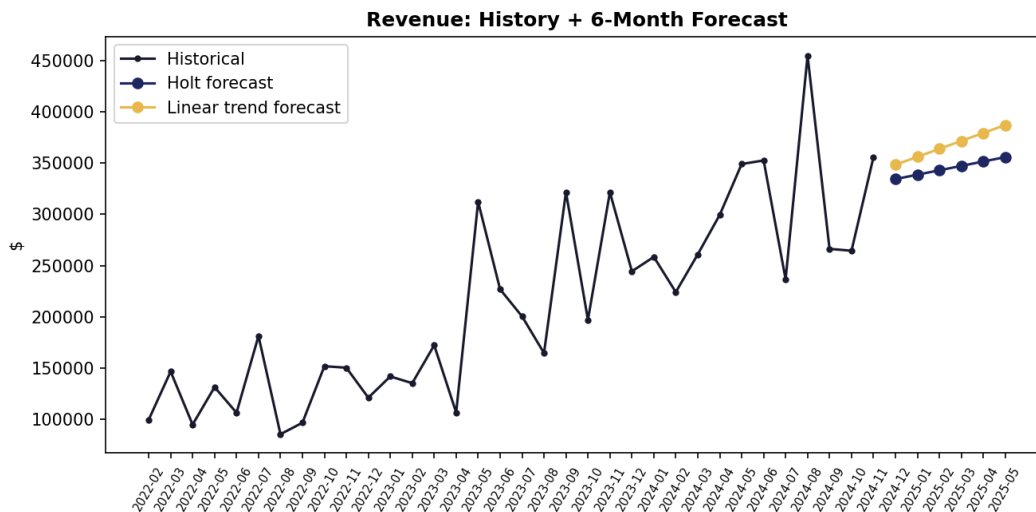


Figure 6.8 — Historical monthly revenue plus 6-month forecasts from both models.

Model		In-sample MAE	In-sample RMSE
Holt's linear		\$56,990.39	\$72,524.81
Linear trend (OLS)		\$42,291.66	—
Month	Holt forecast		Linear trend forecast
2024-12	\$334,411.01		\$348,389.54
2025-01	\$338,699.6		\$356,142.52
2025-02	\$342,988.18		\$363,895.5
2025-03	\$347,276.77		\$371,648.48
2025-04	\$351,565.35		\$379,401.45
2025-05	\$355,853.94		\$387,154.43

Both models agree on direction — revenue rising — but diverge by month 6 (\$355,853.94 vs. \$387,154.43). That gap is the trend-extrapolation risk inherent to a fairly short history with no clear seasonal signal; it's exactly why Section 6.5's Monte Carlo range is presented as the primary planning number, with these two point forecasts as supporting context on the near-term trajectory.

7. Limitations & Data Quality Caveats

A number of findings in this report point to characteristics of the dataset itself that any reader relying on it for business decisions should understand:

- Order completion rate (57%) is unusually low, with cancelled/returned/pending volumes nearly matching completed volume. This may reflect real operational issues, but could equally reflect how the dataset was constructed. Any downstream planning that assumes a higher completion rate would be overstated.
- Cohort retention is essentially flat across all 12 months rather than showing the decay curve typical of real customer cohorts, suggesting order timing may not be genuinely coupled to signup date in this data.
- Every price elasticity and category-level basket-lift estimate came back statistically indistinguishable from zero — a pattern consistent with prices, quantities, and category co-occurrence being independent of each other in the underlying data, which would not hold in a real ecommerce business.
- SKU-level market basket pairs are built on very small shared-order counts (3–5 orders) and are not a reliable basis for a bundling decision at current data volume.
- The 99% catalog-vs-internal price mismatch rate is large enough to be a genuine data-governance issue rather than noise, but this report cannot determine which price source is authoritative without input from the merchandising team.
- CLTV, regression, and Monte Carlo modeling all rely on a single margin-rate assumption (48.8%) averaged across the full catalog; category-specific margins would refine these estimates further.

None of these caveats undermine the cleaning, integration, or methodology work in Phases 1–4 — they are exactly the kind of finding a rigorous exploratory pass is supposed to surface, and are reported here rather than smoothed over.

7.1 Why This Matters for How the Report Should Be Used

A report like this is only as useful as the honesty of its caveats. It would have been easy to present the SKU-level basket pairs in Section 5.5 as a ready-made bundling strategy, or to frame the regression's low R^2 as a modeling limitation rather than a signal about the data itself. Both choices would have made for a more confident-sounding report and a worse one. The standard applied throughout this document is that a result is only presented as a recommendation if the underlying statistic clears a normal significance threshold and the sample size supports it; everything else is labeled as exploratory or inconclusive, explicitly, at the point where it's discussed rather than only in this section.

Practically, this means the reader should treat Sections 6.1, 6.2, and 6.5–6.7 (segmentation, CLTV, Monte Carlo, forecasting) as reasonably load-bearing for planning purposes, and Sections 5.5 and 6.4 (basket analysis, elasticity) as directional at best until re-validated on more data.

8. Business Recommendations

8.1 Prioritized actions

- Reactivate the "At Risk (High Value)" segment first (210 customers, avg CLTV \$2,734.09) — the highest-value-per-customer group that has gone quiet, and the clearest ROI target identified in this analysis.
- Resolve the catalog-vs-internal pricing discrepancy with the merchandising team before either price source is used operationally — a 99% mismatch rate is a systems issue, not a modeling one.
- Investigate the 57% order-completion rate and the unusually low variance in cancelled/returned/pending volumes as an operational question, independent of this analytics work.
- Hold off on any basket-bundling or elasticity-based pricing action until either transaction volume grows meaningfully or the flat elasticity/lift results are validated against a second data source.
- Use the Monte Carlo P10–P90 range (\$2.13M–\$2.98M) for 12-month revenue planning rather than either single-point forecast model in isolation.

8.2 Suggested follow-up analysis

- Re-run cohort retention and elasticity analysis once a larger or more recent data extract is available, to determine whether the flat patterns found here are structural or a data-volume artifact.
- Segment CLTV and margin rate by product category rather than using a single blended rate.
- Extend the recommendation engine with a held-out evaluation set (e.g. precision@k) once enough repeat-purchase data accumulates to measure it meaningfully.

9. Conclusion

This report took UrbanCart's operational database and two supporting exports through a complete four-phase analytics pipeline: SQL-based profiling and validation, cleaning and cross-source reconciliation, exploratory analysis of customer and purchase behavior, and a set of from-scratch NumPy models covering segmentation, lifetime value, regression, elasticity, simulation, recommendation, and forecasting.

The most actionable findings are concrete and immediately usable: a well-defined At-Risk-High-Value customer segment worth targeted reactivation spend, a systematic and previously invisible pricing discrepancy between two data sources, and a defensible revenue range for the next 12 months. Equally important are the negative findings — flat price elasticity, near-chance market basket lift, and a low-explanatory-power spend regression — which are reported plainly rather than reframed as something more conclusive than the data supports. Both kinds of finding are, deliberately, held to the same standard of evidence throughout this report.

Appendix A — Full RFM Segment Detail

Segment	Customers	Avg. recency (days)	Avg. frequency	Avg. monetary (\$)	Avg. CLTV (\$)
Loyal Customers	707	179.07	2.52	3,327.93	1,698.55
Hibernating/Lost	535	988.07	0.45	280.6	391.64
Champions	441	78.68	3.77	6,934.79	3,257.53
Needs Attention	416	496.25	1.32	2,878.77	1,953.30
At Risk (High Value)	210	561.61	2.86	4,192.84	2,734.09
New/Promising	191	99.77	1.07	1,350.83	575.71

Appendix B — Full Price Elasticity Detail

Category	Elasticity	t-stat	R ²	n
Apparel	0.0115	1.4	0.0011	1862
Beauty & Health	-0.005	-0.646	0.0002	1866
Books & Media	0.0051	0.846	0.0004	1903
Electronics	-0.0096	-1.121	0.0007	1832
Home & Kitchen	-0.0121	-1.32	0.0009	1914
Sports & Outdoors	-0.0003	-0.032	0	1815
__overall__	-0.0002	-0.062	0	11192

Appendix C — Cohort Retention, Selected Cohorts

A representative sample of monthly cohorts and their retention rate at each month offset (0–11), drawn from the full matrix in `outputs/cohort_retention_matrix.csv`. A dash indicates zero customers from that cohort placed a completed order in that month (which collapses to the same blank cell as a cohort not yet reaching that month offset, since both cases produce no matching rows in the underlying SQL group-by).

Cohort	M0	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11
2021-02	—	—	—	—	—	—	—	—	—	—	—	6%
2021-05	—	—	—	—	—	—	—	—	3%	6%	—	3%
2021-08	—	—	—	—	—	13%	3%	8%	5%	5%	11%	11%
2021-11	—	—	11%	7%	5%	9%	5%	2%	7%	7%	2%	16%
2022-02	3%	6%	6%	12%	3%	9%	9%	6%	3%	9%	6%	12%
2022-05	—	9%	13%	2%	2%	2%	4%	7%	2%	2%	11%	—
2022-08	5%	13%	10%	18%	18%	10%	3%	8%	13%	3%	5%	5%
2022-11	2%	14%	12%	10%	2%	2%	12%	2%	7%	7%	10%	10%
2023-02	3%	13%	13%	10%	10%	10%	10%	6%	10%	16%	6%	10%
2023-05	6%	3%	16%	3%	16%	16%	6%	6%	19%	3%	—	6%
2023-08	9%	9%	15%	15%	12%	6%	12%	24%	9%	9%	9%	9%
2023-11	9%	14%	19%	16%	7%	14%	9%	21%	7%	21%	9%	16%

Appendix D — Full Category-Level Affinity Table

All 15 category pairs from Section 5.5, ranked by lift. Every value is ≤ 1 , meaning no category pair sells together more than random chance would predict at the order level.

Category A	Category B	Pair orders	Support	Lift
Apparel	Sports & Outdoors	452	0.0896	0.9373
Beauty & Health	Sports & Outdoors	434	0.0861	0.8887
Books & Media	Electronics	437	0.0867	0.8686
Electronics	Sports & Outdoors	418	0.0829	0.8685
Home & Kitchen	Sports & Outdoors	429	0.0851	0.8633
Books & Media	Sports & Outdoors	426	0.0845	0.8621
Beauty & Health	Books & Media	440	0.0872	0.8619
Beauty & Health	Home & Kitchen	434	0.0861	0.8455
Books & Media	Home & Kitchen	437	0.0867	0.8413
Apparel	Books & Media	423	0.0839	0.8392
Electronics	Home & Kitchen	424	0.0841	0.8381
Beauty & Health	Electronics	408	0.0809	0.8206
Apparel	Beauty & Health	405	0.0803	0.813
Apparel	Home & Kitchen	412	0.0817	0.8128
Apparel	Electronics	398	0.0789	0.8107

Appendix E — Reproducibility & File Manifest

Every figure, table, and chart in this report is generated by the accompanying code and is fully reproducible from the two source files (ecommerce.db, plus the two CSV exports).

Repository structure

- queries.sql — all Phase 1 SQL, runnable standalone: `sqlite3 ecommerce.db < queries.sql`
- analysis.ipynb — Phases 1–4 in one notebook, clearly section-labeled, executed top to bottom with no errors
- data/processed/clean_orders.csv, clean_customers.csv — the two integrated Phase 2 deliverables
- report.pdf — this document
- executive_summary.pdf — one-page non-technical summary
- README.md — project overview and contribution statement

How to reproduce

1. Place ecommerce.db, legacy_customers_export.csv, and product_catalog_2024.csv alongside the repository files (or update the path constants at the top of the notebook).

2. `pip install pandas numpy rapidfuzz jupyter`

3. `jupyter nbconvert --to notebook --execute --inplace analysis.ipynb`

This regenerates data/processed/*.csv and every number reported above.